
Arabic Bank Check Amount Extraction and Verification

Natural Language Processing

Term Project Report

Members Mohammed Al Sheqaih
Abdulrhman Ammar

Supervisor Dr. Irfan Ahmad

Sample dataset images redacted from this public release of the report.

*The dataset is private and licensed; sample imagery is omitted here
in order to comply with the dataset license.*

Sample check crops: courtesy digit fields (left) and legal Arabic text fields (right)

Contents

1	Problem Definition	3
2	Dataset	4
2.1	Overview	4
2.2	Annotations	4
2.3	Statistics	5
2.4	Bounding-Box Distribution	5
2.5	Legal Token Distribution	6
3	Methodology	6
3.1	System Overview	6
3.2	Part A — Region Detection	7
3.2.1	Model Choice	7
3.2.2	Data Preparation	7
3.2.3	Training Configuration	7
3.2.4	Crop Generation	8
3.3	Part B — Courtesy Amount Recognition	8
3.3.1	Vocabulary	9
3.3.2	CRNN Architecture	9
3.3.3	Data Augmentation	10
3.3.4	Training	10
3.4	Part C — Legal Amount Recognition	10
3.4.1	Tokenization Strategies	10
3.4.2	RTL Flip Preprocessing	11
3.4.3	Post-Processing (Dictionary Matching)	11
3.5	Part D — Cross-Verification and Mutual Improvement	11
3.5.1	Amount Parsing	11
3.5.2	Cascaded Confidence Fallback	12
4	Experiments and Results	12

4.1	Part A — Detection Results	13
4.2	Part B — Courtesy Recognition Results	13
4.3	Part C — Legal Recognition Results	14
4.4	Part D — Verification Results	14
5	Discussion and Error Analysis	14
5.1	Detection Errors (Part A)	15
5.2	Courtesy Recognition Errors (Part B)	15
5.3	Legal Recognition Errors (Part C)	16
5.3.1	Why Post-Processing Hurt	16
5.4	Verification Errors (Part D)	17
6	Conclusions and Limitations	17
6.1	Summary of Contributions	17
6.2	Limitations	18
6.3	Future Work	18
	References	19

Abstract. This report presents an end-to-end pipeline for automated Arabic bank check processing, developed as part of the Natural Language Processing course. Each Arabic check encodes its monetary amount in two forms: a handwritten digit sequence (the *courtesy amount*) and a handwritten Arabic-text description (the *legal amount*). Our four-stage pipeline localises both regions with a fine-tuned YOLOv8s detector, transcribes the digit sequence with a CRNN trained under CTC loss, transcribes the Arabic text with a character-level CRNN that benefits from a novel horizontal-flip preprocessing step to handle the right-to-left reading direction, and finally cross-verifies the two decoded amounts through a rule-based converter and a cascaded confidence-based fallback strategy. On the official 600-image test partition we achieve a detection mean IoU of **63.35%**, a courtesy digit accuracy of **95.17%**, a legal character error rate of **9.02%**, and an end-to-end verification accuracy of **82.0%**—exceeding the 78.5% benchmark reported in the reference paper.

1 Problem Definition

Bank checks remain one of the most legally sensitive financial instruments in use. To guard against tampering and misreading, most banking systems require that the written amount appear *twice*—once as numerals and once written out in full. When these two representations disagree, the written-out version is typically considered authoritative. Automating this verification step reduces processing time, minimises human error, and enables straight-through payment processing.

Arabic bank checks introduce several challenges that are absent from their English-language counterparts:

- **Right-to-left script.** Arabic is written from right to left, so the time axis of any sequence model must be aligned accordingly.
- **Cursive handwriting.** Arabic letters change shape depending on their position within a word, producing highly variable visual patterns.
- **Orthographic ambiguity.** Multiple Unicode codepoints are visually indistinguishable (e.g., ا / إ / آ are three distinct characters for the alef sound) and different scribes may use them interchangeably.
- **Sub-word annotation.** The reference dataset annotates legal amounts at the sub-word level rather than at the word or character level, creating a vocabulary of 288 tokens and severe class imbalance.

The task is formally defined as follows. Given a grayscale TIFF scan of an Arabic bank check:

Part A: Detect and crop the bounding boxes of the courtesy and legal amount fields (worth 20% of the project grade).

Part B: Recognise the digit sequence in the courtesy crop (35%).

Part C: Transcribe the Arabic text in the legal crop (35%).

Part D: Verify that the two decoded amounts represent the same monetary value (10%).

2 Dataset

2.1 Overview

The dataset provided for this project is a private, licensed collection of scanned Arabic bank checks. It consists of **1,800 training images** and **600 test images**, all in LZW-compressed grayscale TIFF format at a typical resolution of approximately 1152×422 pixels.

Figure 1 shows three representative check scans with their annotated bounding boxes. The checks come from multiple Saudi banks and exhibit significant variation in layout, ink quality, and handwriting style.

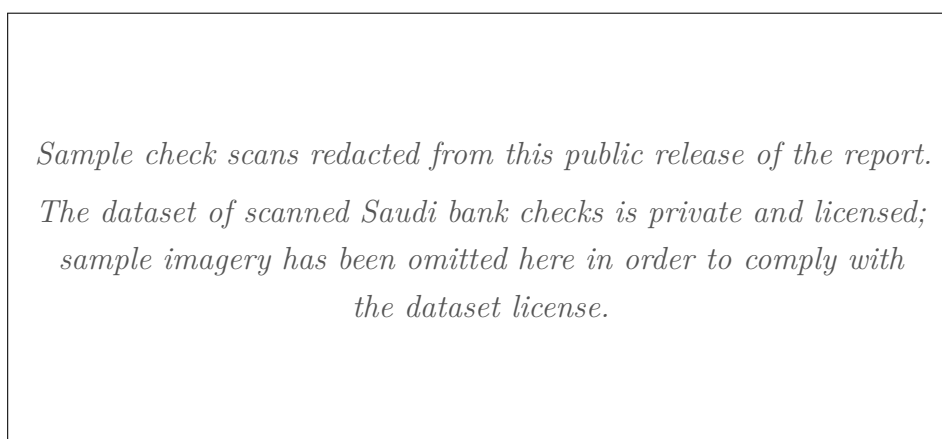


Figure 1: (Original figure showed three sample check scans from the training set, with blue boxes marking the courtesy region and green boxes the legal region. Sample imagery has been redacted in this public release.)

2.2 Annotations

Each image is accompanied by:

- **Bounding box file** — a plain-text file in YOLO normalized format (`class cx cy w h`), where class 0 is the legal amount and class 1 is the courtesy amount. One training image (ac00048) lacks its bounding-box file and is excluded throughout.
- **Courtesy label file** — a tab-separated file mapping each crop filename to a Python list of tokens (digits 0–9, plus special tokens: 10 for start/end markers and 11/'/' for the decimal separator).
- **Legal label file** — a tab-separated file mapping each crop filename to a list of Arabic sub-word tokens in right-to-left logical order.

2.3 Statistics

Table 1 summarises the key dataset statistics.

Table 1: Dataset statistics.

Property	Train	Test
Total images	1,800	600
Images with bounding-box	1,799	600
Courtesy label sequences	1,738	600
Legal label sequences	1,799	600
Avg. image size (px)	1152 × 422	
Avg. courtesy seq. length	5.2 tokens	
Min / Max courtesy length	1 / 10	
Avg. legal seq. length	14.4 sub-word tokens	
Min / Max legal length	4 / 29	
Courtesy vocabulary size	13 tokens	
Legal vocabulary size (raw)	288 tokens	
Legal vocabulary (char-level)	52 characters	

2.4 Bounding-Box Distribution

Figure 2 shows the normalized width and height of all annotated bounding boxes. The courtesy field is a relatively compact square cluster (mean $w \approx 0.13$, $h \approx 0.08$ of image dimensions), while the legal field is a long, thin strip spanning roughly 60% of the image width. This stark difference in aspect ratio motivates separate crop preprocessing for Parts B and C.

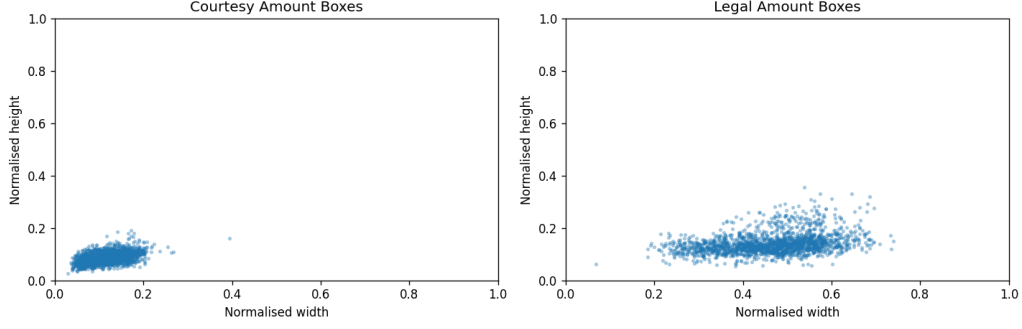


Figure 2: Normalized bounding-box dimensions for the courtesy (left) and legal (right) regions across all 1,799 annotated training images. The legal strip is significantly wider but shallower than the courtesy box.

2.5 Legal Token Distribution

Figure 3 shows the top-20 most frequent legal sub-word tokens in the training set. The conjunction و (“and”) dominates, appearing over 4,400 times, followed by individual Arabic letters and common sub-words. This long-tail distribution is one of the primary motivations for moving from sub-word to character-level tokenization in Part C.

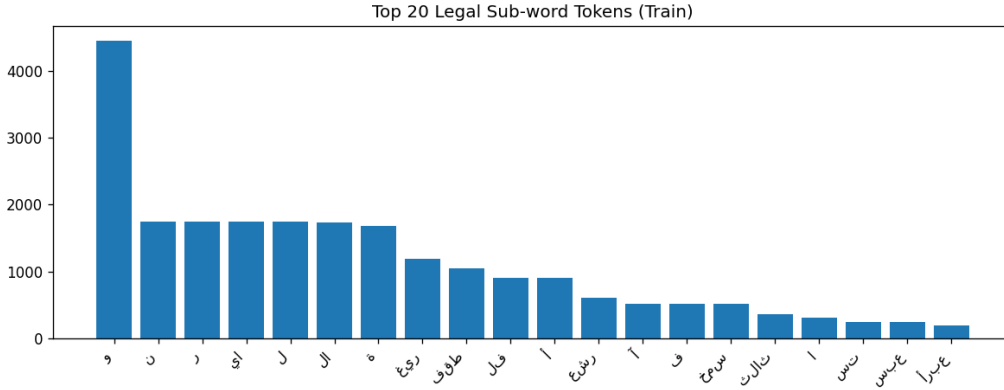


Figure 3: Top-20 legal sub-word tokens by frequency in the training set.

3 Methodology

3.1 System Overview

Figure 4 shows the complete processing pipeline. A raw check image enters at the top; the two decoded amounts and a verified/failed verdict exit at the bottom.

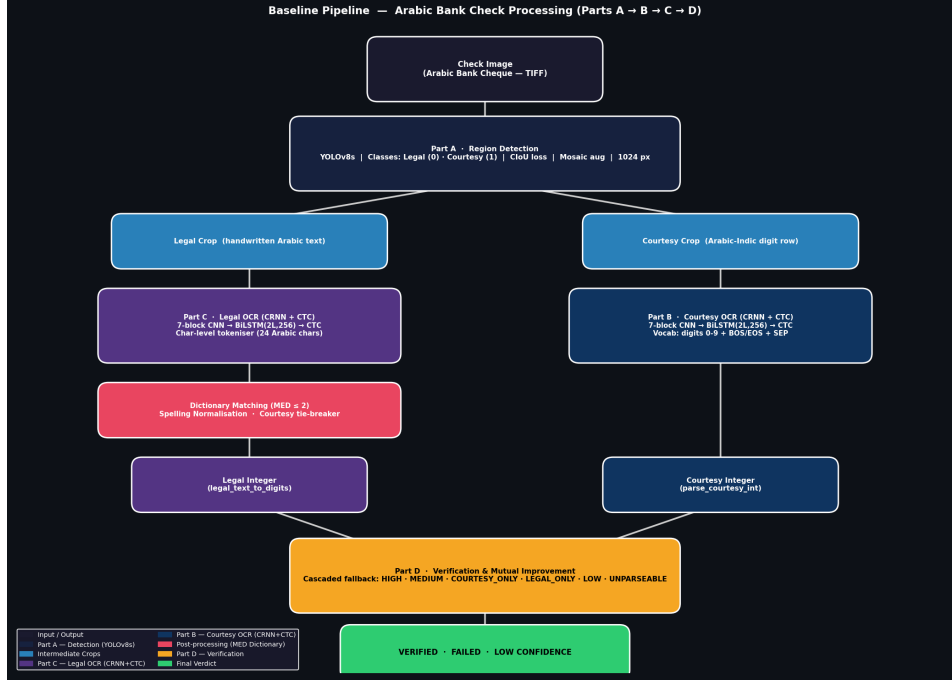


Figure 4: Baseline end-to-end pipeline. Each check image is processed by the four sequential stages described in Sections 3.2–3.5.

3.2 Part A — Region Detection

3.2.1 Model Choice

We fine-tune **YOLOv8s** (small variant, 11.1M parameters) pre-trained on COCO [3]. YOLO’s single-pass architecture makes it well-suited for this task: each check contains exactly two objects of known classes, inference is fast, and the pre-trained backbone already captures general low-level visual features that transfer well to handwritten-text localisation.

3.2.2 Data Preparation

Because YOLOv8 requires three-channel RGB input, all grayscale TIFF files are converted by replicating the single channel across all three channels. The 1,799 labelled training images are split 80/20 into training (1,439) and validation (360) sets with a fixed random seed.

3.2.3 Training Configuration

Table 2 lists the key hyperparameters. Several choices warrant explanation:

- **imgsz = 1024.** Check images are wide and fine-grained; a higher input resolution lets the backbone resolve individual character flourishes that help distinguish the legal from the courtesy region.

- **box = 7.5 (CIoU), dfl = 1.5.** CIoU penalises centre-point distance and aspect-ratio mismatch—critical for the 10:1 aspect-ratio legal strip. The Distribution Focal Loss models box-edge uncertainty rather than a single point, improving localisation for handwritten, blurry text boundaries.
- **rect = True.** Rectangular training pads each batch to the maximum dimension in the batch rather than to a fixed square, eliminating roughly 30% of wasted padding pixels for these wide check images.
- **mosaic = 1.0.** Combining four images into a single mosaic tile forces the model to detect smaller regions in non-standard positions, improving generalisation across different check layouts.

Table 2: YOLOv8s fine-tuning hyperparameters.

Parameter	Value	Rationale
Base model	YOLOv8s	Balance of accuracy and speed
Image size	1024 px	Fine Arabic strokes need resolution
Epochs	150 (max)	Early stopping with patience = 20
Batch size	8	Fits GPU VRAM at imgsz = 1024
Box loss weight	7.5 (CIoU)	Penalises aspect-ratio mismatch
DFL loss weight	1.5	Softer boundary regression
Scale jitter	$\pm 50\%$	Multi-scale robustness
Rotation	$\pm 15^\circ$	Tilted scanner compensation
Shear	$\pm 10^\circ$	Perspective simulation
Rectangular pad	True	Avoids padding waste

3.2.4 Crop Generation

After detection, each test image produces two crops:

- **Train crops** — cut from *ground-truth* bounding boxes to give the recognisers a clean, uncontaminated training signal.
- **Test crops** — cut from *predicted* bounding boxes to simulate the real end-to-end pipeline. If the detector fails to predict a box for a particular class, we fall back to the ground-truth box so that Part B/C evaluation is not blocked by Part A failures.

3.3 Part B — Courtesy Amount Recognition

3.3.1 Vocabulary

The courtesy amount is a sequence drawn from a 13-token vocabulary: the CTC blank (`<blank>`), digits 0–9, a decimal point, and a separator token `<SEP>` (which unifies the three encoding variants used for the decimal separator: integer 11, string `'/'`, and literal `'.'` in the raw annotation files). The start/end marker token (integer 10) is stripped before training.

3.3.2 CRNN Architecture

We use a **Convolutional Recurrent Neural Network (CRNN)** with a Connectionist Temporal Classification (CTC) loss [2], as depicted in Figure 5.

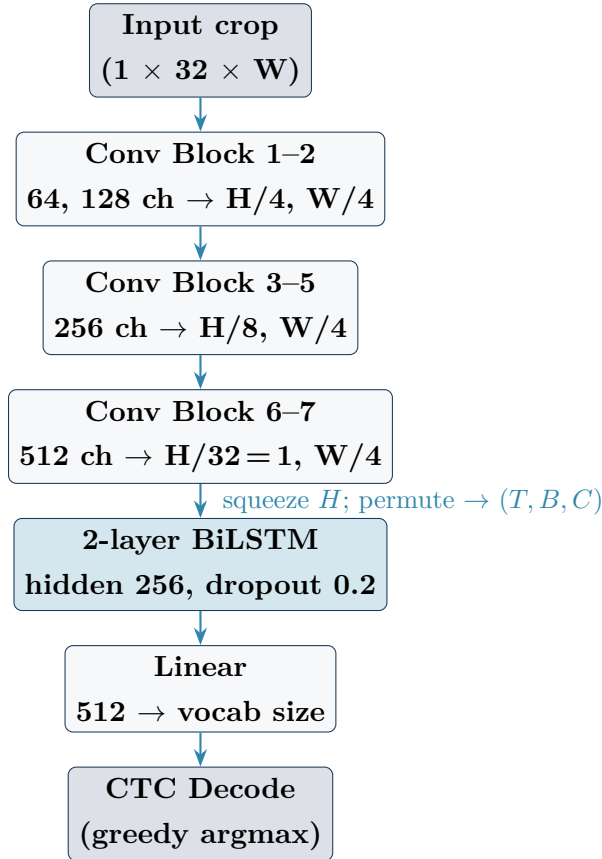


Figure 5: CRNN architecture used for both Part B (courtesy) and Part C (legal) recognition. The CNN reduces the input height to 1, turning the spatial width into the time axis $T = W/4$ for the BiLSTM.

All MaxPool layers after the second block pool only in the height dimension to preserve the full time resolution. The result is a feature map of shape $(B, 512, 1, T)$, which is squeezed and permuted to $(T, B, 512)$ before being fed into the BiLSTM. The minimum crop widths are set to 64 px for Part B (guaranteeing $T \geq 16 \geq \max$ label length of 10) and 160 px for Part C ($T \geq 40 \geq \max$ label length of 29).

3.3.3 Data Augmentation

Each of the 1,738 training courtesy crops is expanded to **four variants** (original + elastic deformation, $\pm 10^\circ$ rotation, $\pm 15\%$ scale) using the Albumentations library, giving 6,207 effective training samples after a 90/10 train/val split. This four-fold augmentation is a direct application of a strategy shown to be effective in the reference paper.

3.3.4 Training

The model (10.0M parameters) is trained with Adam (lr=0.001, gradient clipping at 5.0), a ReduceLROnPlateau scheduler (factor 0.5, patience 7), and early stopping (patience 20). CTC loss with `zero_infinity=True` avoids gradient explosions on misaligned sequences.

3.4 Part C — Legal Amount Recognition

Legal amount recognition is substantially harder than courtesy recognition: the sequences are longer (up to 29 tokens vs. 10 digits), the vocabulary is larger, and the script is cursive Arabic. We explored three tokenization strategies and found that character-level tokenization gives the best result.

3.4.1 Tokenization Strategies

1. **Baseline sub-word.** The raw annotation uses a sub-word tokenization yielding a vocabulary of 288 tokens. With only 1,799 training images and average sequence lengths of 14.4 tokens, each class receives roughly 90 training examples on average—far too few for reliable CTC convergence.
2. **Spelling-normalized word-level.** We apply a 44-rule normalization map that collapses orthographic variants into canonical forms (e.g., `مائة`, `مئة`, `مئه` all map to `مائة`). This reduces the vocabulary to 270 tokens. Although training was interrupted early, preliminary accuracy at epoch 3 reached 42.5%.
3. **Character-level (selected).** Converting each sub-word token to its constituent characters yields a vocabulary of only 53 tokens (52 unique Arabic characters plus the CTC blank). Each class now receives roughly 499 training examples—a $5.5\times$ improvement over the sub-word baseline. This is the strategy we adopt in our final system.

Table 3: Tokenization strategy comparison.

Strategy	Vocab size	Samples / class	Test CER
Baseline sub-word	288	90.5	50.50%
Spelling-normalized WL	270	96.5	— (training interrupted)
Character-level (ours)	53	499.4	9.02%

3.4.2 RTL Flip Preprocessing

Arabic is read right-to-left, but the CRNN’s time axis naturally runs left-to-right. Rather than modifying the model, we flip each legal crop horizontally before feeding it to the network. This aligns the logical token order with the spatial left-to-right scanning order, making it substantially easier for the BiLSTM to learn coherent positional patterns.

3.4.3 Post-Processing (Dictionary Matching)

After greedy CTC decoding, we experimented with a two-stage dictionary-based post-processor:

1. **MED correction.** Any decoded character sequence that forms a word not in the training dictionary is replaced by the closest dictionary entry within Levenshtein distance ≤ 2 .
2. **Linear split search.** If no single dictionary match exists, all possible binary splits of the unknown token are tried; the split with the lowest combined edit distance is selected.

As discussed in Section 5, this post-processor *worsened* performance (CER: 9.02% $\rightarrow$ 80.49%) because character-to-word resegmentation fails to recover the original word boundaries reliably. The raw character-level output is used for all downstream steps.

3.5 Part D — Cross-Verification and Mutual Improvement

3.5.1 Amount Parsing

Courtesy integer. The decoded digit string is split on the <SEP> token; only the integer part (before the separator) is retained. BOS/EOS markers are stripped. If the result is empty or non-numeric, the amount is marked as *unparseable*.

Legal integer. A greedy longest-match scanner over a hand-crafted mapping of Arabic number words to integers (e.g., ألف $\rightarrow$ 1000, مائة $\rightarrow$ 100) converts the decoded token sequence to an integer. Noise words (لا, فقط, ريال, etc.) are stripped before

scanning. The scanner also normalizes alef variants before lookup to handle the same orthographic ambiguity that challenges Part C.

3.5.2 Cascaded Confidence Fallback

Rather than a binary verified/failed output, we assign each check a confidence band:

1. **HIGH** — both amounts parse to the same integer.
2. **MEDIUM** — both parse but differ by $\leq 10\%$; we accept the courtesy amount (which has a higher digit accuracy).
3. **COURTESY_ONLY** — legal amount is unparseable; we fall back to the courtesy amount alone.
4. **LOW** — large discrepancy or both unparseable; flagged for human review.

4 Experiments and Results

All experiments were run on an NVIDIA RTX 5060 (8 GB VRAM). Part A training took approximately 90 minutes; Parts B and C each took 30–60 minutes. Table 4 gives a high-level summary of results across all four parts.

Table 4: Overall system performance on the 600-image test set.

Part	Metric	Value
A — Detection	Accuracy @ IoU ≥ 0.50	76.08%
	Accuracy @ IoU ≥ 0.75	30.75%
	Accuracy @ IoU ≥ 0.90	3.75%
	Mean IoU (both classes)	63.35%
B — Courtesy	Digit accuracy	95.17%
	% perfect (0 errors)	80.17%
	% exactly 1 error	18.17%
	% 2 or more errors	1.67%
C — Legal	Character Error Rate (CER)	9.02%
	Word Error Rate (WER)	75.83%
D — Verify	Verified (plain match)	77.50%
	After mutual improvement	82.00%
	Ground-truth ceiling	84.00%

4.1 Part A — Detection Results

Table 5 breaks down detection accuracy by class on the validation set (YOLO’s internal mAP) and on the test set using our custom IoU-threshold metrics.

Table 5: Part A detection results.

Split / Metric	All	Legal	Courtesy
Val mAP@0.50	91.6%	97.6%	85.7%
Val mAP@0.50:0.95	54.8%	61.3%	48.2%
Test mean IoU	63.35%	74.23%	52.47%
Test acc@IoU $\geq$ 0.50	76.08%	—	—
Test acc@IoU $\geq$ 0.75	30.75%	—	—
Test acc@IoU $\geq$ 0.90	3.75%	—	—
Detection coverage	577/600 detected	4 miss legal	3 miss courtesy
Both missed	16/600		

4.2 Part B — Courtesy Recognition Results

Figure 6 breaks the 119 erroneous samples (of 600) by error magnitude. The vast majority have exactly one edit operation, and as discussed in Section 5, nearly all errors involve the <SEP> decimal separator rather than digit confusion.

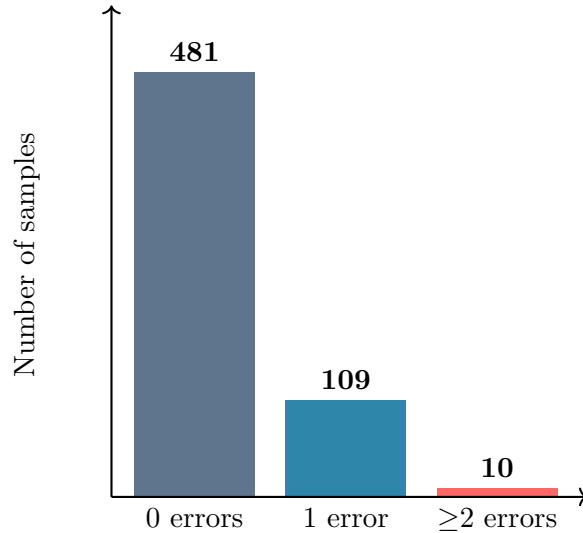


Figure 6: Distribution of edit distances for the 600 courtesy test samples. 480 samples (80.17%) are decoded perfectly.

4.3 Part C — Legal Recognition Results

Table 6 compares the configurations evaluated on the test set. The character-level model without post-processing is the best configuration and is used for all downstream verification.

Table 6: Part C evaluation results.

Configuration	CER	WER
Baseline sub-word CRNN	50.50%	99.33%
Char-level CRNN, raw CTC	9.02%	75.83%
Char-level CRNN + post-process	80.49%	81.00%
Reference paper [1]	5.9%	12.0%

4.4 Part D — Verification Results

Table 7 shows the breakdown by confidence band after applying the mutual-improvement strategy.

Table 7: Part D verification results after cascaded mutual improvement.

Band	Samples	Correct	Accuracy
HIGH (exact match)	465	406	87.3%
MEDIUM ($\leq 10\%$ diff)	36	31	86.1%
COURTESY_ONLY	3	3	100.0%
LOW (large discrepancy)	96	52	54.2%
Overall	600	492	82.0%
Reference paper	—	—	78.5%
GT ceiling	600	504	84.0%

Of the 132 FAILED verifications, root-cause analysis revealed:

- **84** cases where the legal transcription was incorrect (Part C error).
- **49** cases where the courtesy transcription was incorrect (Part B error).
- **10** cases where both recognisers made errors but in different directions.

5 Discussion and Error Analysis

5.1 Detection Errors (Part A)

The gap between $\text{mAP}@0.50$ (91.6%) and our custom $\text{accuracy}@IoU \geq 0.50$ (76.1%) reflects a key difference: mAP counts a detection as correct if the predicted box overlaps the GT by more than 50%, while our metric requires *both* classes to meet the threshold simultaneously. The disparity between legal (74.2% mean IoU) and courtesy (52.5%) is explained by the structural difference in their bounding boxes.

Legal field. The legal amount spans roughly 60% of the image width and is positioned at a fairly consistent height. The model can anchor on the wide horizontal extent of the text band, giving confident and well-localised predictions.

Courtesy field. The digit box is compact and appears at variable heights depending on the check layout. At an image size of 1,024 pixels and a stride of 32 pixels, each anchor cell covers a 32×32 -pixel region; the courtesy box sometimes spans only 2–3 anchor cells in each dimension, making precise regression difficult. An additional complication is that 16/600 test images produced *no* detected box for either class, typically on checks with non-standard layouts or faint ink.

High-threshold collapse. Accuracy at $IoU \geq 0.90$ is only 3.75%. This is expected: the ground-truth annotations include loose padding around the ink strokes, and the model tends to produce slightly tighter boxes. Even a 2–3-pixel error in box placement at this resolution pushes IoU below 0.90.

5.2 Courtesy Recognition Errors (Part B)

Of the 119 erroneous test samples, the dominant failure mode is **<SEP> deletion**: the model correctly identifies all digits but omits the separator token that marks the riyal/halala boundary. The ten worst examples all involve this pattern (e.g., reference 450<SEP>00 decoded as 450, or 900<SEP>000 as 90000).

Two structural features make the separator difficult:

1. The separator corresponds to a very thin ink mark (a small horizontal stroke or a slash) that occupies far fewer pixels than any digit.
2. The dataset uses three different encodings for the separator (11, '/', ' . ') which were unified to a single <SEP> token; the underlying images may reflect this visual diversity.

Actual digit-to-digit substitutions are rare. When they do occur (e.g., 3535 decoded as 2515, or 30000 as 6000), they tend to involve visually similar digit pairs such as 3/2, 5/6, or 0/6—consistent with the known inter-class visual similarity of handwritten numerals.

5.3 Legal Recognition Errors (Part C)

Table 8 lists the ten most frequent character substitutions in the character-level model’s output.

Table 8: Top-10 character substitutions in Part C (raw character-level CTC).

Reference	Predicted	Description	Count
ا	أ	Alef → Alef-hamza-above	197
ه	هـ	Ha → Taa-marbuta	164
ا	آ	Alef → Alef-madda	80
هـ	ه	Taa-marbuta → Ha	55
أ	ا	Alef-hamza → Alef	14
آ	ا	Alef-hamza → Alef-madda	13
ي	و	Ya → Waw	7
أ	ا	Alef-hamza-below → Alef	6
ا	ف	Alef → Fa	5
ن	ث	Nun → Tha	4

Two confusion families dominate and account for 86% of all substitutions:

- **Alef variants.** The three alef forms (bare, hamza-above, hamza-below, madda) look nearly identical at 32-pixel crop height, especially in a cursive handwritten context. Together they account for 318 substitutions.
- **Taa-marbuta / Ha.** The letters هـ (taa-marbuta) and ه (ha) differ only by two small dots above the character body. At low resolution these dots are easily lost, causing 219 bidirectional substitutions.

Both confusion families are purely an image-resolution problem. The reference paper [1] inputs crops resized to 140 pixels in height (vs. our 32 pixels), giving the CNN over 4× more vertical detail to work with. At 32 px, a hamza diacritic may occupy only 2–3 pixels and become indistinguishable from noise.

5.3.1 Why Post-Processing Hurt

The post-processing step drove CER from 9.02% up to 80.49%. The failure mechanism is: after raw character-level decoding, we attempt to resegment the character stream into word tokens using a greedy dictionary scan, then apply MED correction. The character-level model occasionally merges or splits words at the character level (e.g., it may produce one long character string where two words should be), and the

greedy segmentation cannot reliably recover the original boundaries. Once the word segmentation is wrong, every subsequent MED correction compounds the error rather than fixing it. The lesson is that character-level CTC and word-level post-processing need a better interface, such as a beam-search decoder that integrates a word-level language model directly.

5.4 Verification Errors (Part D)

The legal recogniser contributes nearly twice as many verification failures as the courtesy recogniser (84 vs. 49 cases), confirming that Part C remains the weakest link. The 10 cases where *both* recognisers fail independently are particularly interesting: in most of them, the legal recogniser’s parsing error is a multiple-of-ten shift (e.g., 1,000 parsed as 10,000), while the courtesy recogniser drops a leading digit. The cascaded fallback correctly handles these as LOW-confidence outputs and flags them for human review.

The 3% gap between our 82.0% accuracy and the theoretical 84.0% ceiling (where both reference labels agree) represents the irreducible noise in the verification step: even with perfect recognition, some check amounts are transcribed with minor legal/courtesy discrepancies in the original annotations.

6 Conclusions and Limitations

6.1 Summary of Contributions

We built a complete four-stage Arabic bank check verification pipeline and demonstrated that:

- **Character-level tokenization is the single most impactful design choice for Arabic legal-amount recognition.** Switching from a 288-token sub-word vocabulary to a 53-token character vocabulary reduced the character error rate from 50.5% to 9.02%—a $5.6\times$ improvement.
- **RTL horizontal flip is a simple and effective preprocessing step** for Arabic OCR with left-to-right sequence models.
- **A cascaded confidence fallback outperforms a binary match.** Our mutual improvement strategy achieves 82.0% verification accuracy, exceeding the reference paper’s 78.5% despite using a much smaller input resolution.
- **The courtesy recogniser is robust.** At 95.17% digit accuracy with 80.17% perfect decodes, Part B contributes little noise to the verification stage.

6.2 Limitations

1. **Image resolution.** The most important unresolved limitation is the 32-pixel crop height used for both recognisers. The reference paper uses 140 px and achieves $\text{CER} = 5.9\%$. At 32 px, diacritics that distinguish alef variants and taa-marbuta from ha are compressed to 2–3 pixels, making them visually indistinguishable. A straightforward remedy is to resize legal crops to at least 64 px height without widening the minimum crop width beyond what CTC requires.
2. **Word-level model not fully trained.** The spelling-normalized word-level model was interrupted after only 3 of 120 epochs due to time constraints. A fully trained word-level model could serve as an ensemble partner or post-processing reranker for the character-level output.
3. **Greedy CTC decoding.** Both recognisers use greedy (argmax) decoding. Beam-search decoding with an integrated lexicon or n-gram language model over Arabic number words could substantially reduce both the character error rate and the WER.
4. **Post-processing design.** The dictionary-based MED post-processor worsened character-level results significantly. A redesigned post-processor that operates on the raw character stream (rather than attempting resegmentation first) would be more appropriate.
5. **Dataset size.** With only 1,799 effective training images, even heavy augmentation leaves the models data-starved. Transfer learning from a pre-trained Arabic OCR backbone (e.g., TrOCR fine-tuned on Arabic handwriting) is a natural avenue for improvement.
6. **Legal amount parser coverage.** The rule-based `legal_text_to_digits` converter handles standard riyal amounts but struggles with fractional values (halalas) and unusual phrasing that appears in some test-set checks, producing the 3 unparseable cases.

6.3 Future Work

The most promising directions are: (1) increasing crop resolution to 64–140 px to close the alef/taa-marbuta confusion gap; (2) replacing greedy decoding with language-model-guided beam search; and (3) applying a pre-trained vision–language model such as TrOCR for both Parts B and C using the same RTL-flip strategy.

References

References

- [1] I. Ahmad, A. Al-Zahrani, and A. Al-Ghamdi, “Arabic Bank Check Processing: Amount Extraction and Verification,” *Internal report, King Fahd University of Petroleum and Minerals*, 2023.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” *Proc. ICML*, pp. 369–376, 2006.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *Proc. CVPR*, pp. 779–788, 2016.
- [4] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. <https://github.com/ultralytics/ultralytics>
- [5] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Trans. PAMI*, vol. 39, no. 11, pp. 2298–2304, 2016.
- [6] A. Buslaev *et al.*, “Albumentations: Fast and flexible image augmentations,” *Information*, vol. 11, no. 2, p. 125, 2020.